

```
if(mtd_info)
    put_mtd_device(mtd_info);
}
```

The above kernel module searches for the “userdata” partition. The function `get_mtd_device()`, when invoked with the first argument `NULL`, returns the MTD device associated with the minor number specified in the second argument. On a successful search, it increments the reference count of the device. That’s why, before exiting, a call to `put_mtd_device()` must be made to release (decrement) the reference count.

Additionally, the module uses the flag `MTD_ABSENT` (which is available to applications too). This check is required to function correctly with some probing device drivers used to allocate placeholder MTD devices on systems that have socketed or removable media.

Having retrieved the `mtd_info` structure, reading is relatively simple:

```
/* read something from last sector */
u_char buf[64];
mtd_info->read(mtd_info, mtd_info.size-mtd_info.eraserize,
sizeof(buf), buf);
```

The second argument of the `read` function specifies the read offset, and the third the length to read. Note that the `read` operation too may sleep and, therefore, it must not be performed in an interrupt context. The write operation can be performed as follows (assuming the sector has been previously erased):

```
/* write something to last sector */
mtd_info->write(mtd_info, mtd_info.size-mtd_info.eraserize,
sizeof(buf), buf);
```

As mentioned before, the `read`, `write` and `erase` operations may sleep. Therefore, kernel code must wait for the operation to finish. Here is an example of erasing the partition and waiting to finish the operation:

```
#include <linux/sched.h>

void erase_partition(struct mtd_info *mtd_info) {
    unsigned int start;
    for(start = 0; start < mtd_info->size; start += mtd_info->eraserize)
        erase_sector(mtd_info, start, mtd_info->eraserize);
}

void erase_sector(struct mtd_info *mtd_info, unsigned int start,
unsigned int len) {
    int ret;
    struct erase_info ei = {0};
    wait_queue_head_t waitq;
```

```
DECLARE_WAITQUEUE(wait, current);

init_waitqueue_head(&waitq);
ei.addr = start;
ei.len = mtd_info->eraserize;
ei.mtd = mtd_info;
ei.callback = erase_callback;
ei.priv = (unsigned long)&waitq;
ret = mtd_info->erase(mtd_info, &ei);
if(!ret) {
    set_current_state(TASK_UNINTERRUPTIBLE);
    add_wait_queue(&waitq, &wait);
    if (ei.state != MTD_ERASE_DONE && ei.state != MTD_ERASE_
FAILED)
        schedule();
    remove_wait_queue(&waitq, &wait);
    set_current_state(TASK_RUNNING);

    ret = (ei.state == MTD_ERASE_FAILED)?-EIO:0;
}
}

void erase_callback (struct erase_info *instr) {
    wake_up(&wait_queue_head_t *)instr->priv;
}
```

The `erase_partition()` function iterates over all sectors, and erases them with `erase_sector()`. At the core of `erase_sector()` is the `mtd_info->erase` call, which (as mentioned previously) may sleep. Therefore, `erase_sector()` prepares a wait queue and a wait queue head. After a call to `mtd_info->erase`, the function prepares itself to relinquish the CPU (presuming that `mtd_info->erase` will sleep) by changing task state to `TASK_UNINTERRUPTIBLE` and adding itself to the wait queue head. Before relinquishing the CPU, it checks if erase is done, through the `ei.state` flag. If erase is done successfully, this flag will be set to `MTD_ERASE_DONE`.

If the erase operation is not complete, the task relinquishes the CPU by calling `schedule()`. Later, when the erase operation is complete, the driver calls the callback function provided in `ei.callback`. Here the task wakes up to itself, then removes itself from the wait queue, changes the task state to `TASK_RUNNING` and finally, the `erase_sector()` function returns.

MTD devices have many more features that can be used by application programmers. ECC (error correction codes) and OOB (out of band) data are some of them. The MTD framework is integrated into the Linux kernel—therefore it makes working with MTD devices very simple, as we have seen in this article. 

By: Mohan Lal Jangir

The author is a development manager at Samsung India Software Operations, Bangalore, India. He has a Masters in Computer Technology from IIT Delhi, and is keenly interested in Linux, networking, and network security.